

ANALYSIS & VISUALIZATION OF TIME SERIES DATA IN PYTHON

DAY 2

Dr. –Ing. Mohamed-Ali Tnani

December 2025



What We Learned (Day 1)

◆ Foundations of Time-Series

Trend · Seasonality · Cycles · Noise

Stationarity & differencing

Autocorrelation patterns (ACF / PACF)

◆ Preparing Data for Modeling

Resampling & handling missing values

Normalization

◆ Classical Forecasting

AR, MA, ARIMA

Baseline models (Naive, Seasonal Naive)

Model diagnostics using ACF/PACF



What will we cover?

Day 2

- **Overview on Classical Methods mit Darts Lib**

Prophet

- **Overview on Deep Learning Architectures for Time-Series Data**

RNNs · LSTMs · GRUs · TCNs · Transformer/TFT

- **Training & Evaluation techniques for Time-Series Data**

Metrics · Avoiding leakage · Hyperparameter-Tuning



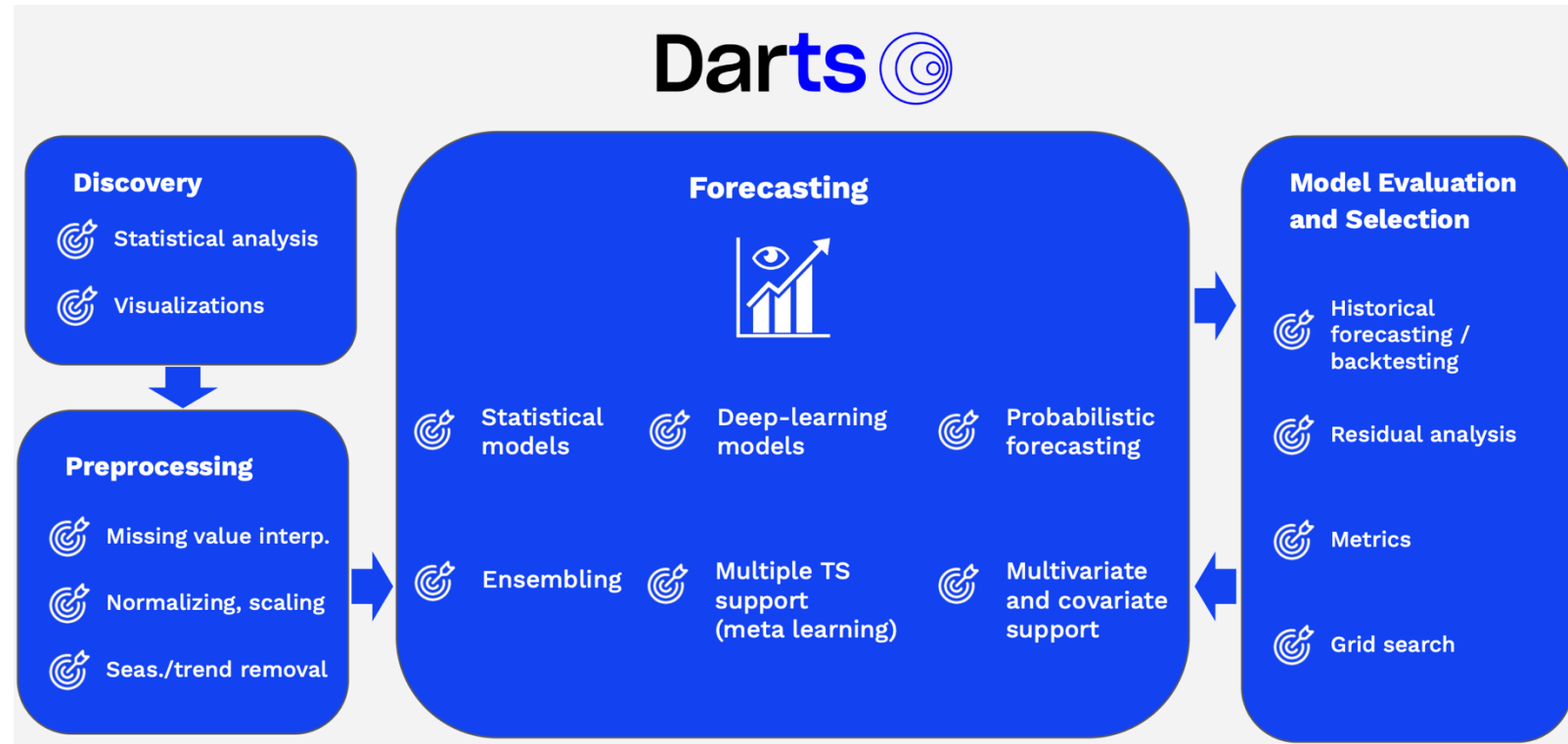
PAUSE

- 10:45 : Coffee Break
- 12:30 – 13:30: Lunch Break



DARTS LIBRARY: A UNIFIED LIBRARY FOR TIME-SERIES FORECASTING

- Built on **PyTorch + LightGBM + scikit-learn + StatsModels**
- Supports **classical, ML, and deep learning** models under one interface
- Designed for **easy experimentation** and **production use**



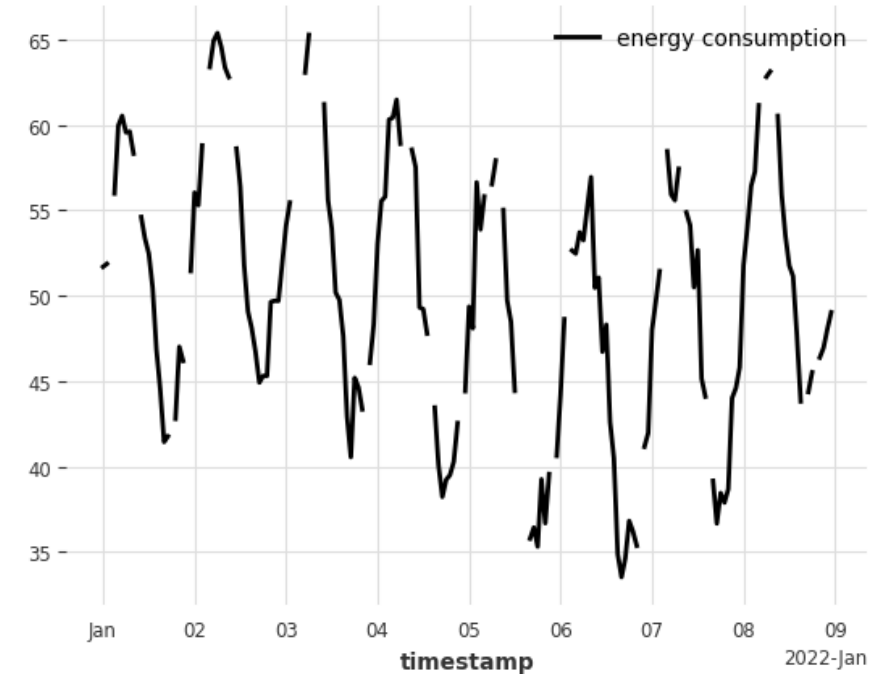
DARTS LIBRARY

```
from darts import TimeSeries

df = pd.read_csv('energy_synthetic.csv')
series = TimeSeries.from_dataframe(df, time_col='timestamp')

series.plot(label='Synthetic Energy data')
```

- **Wrapper** around pandas DataFrame with a **datetime index**
- Handles **single or multiple dimensions (multivariate)**
- Keeps **frequency, metadata, and scaling** consistent



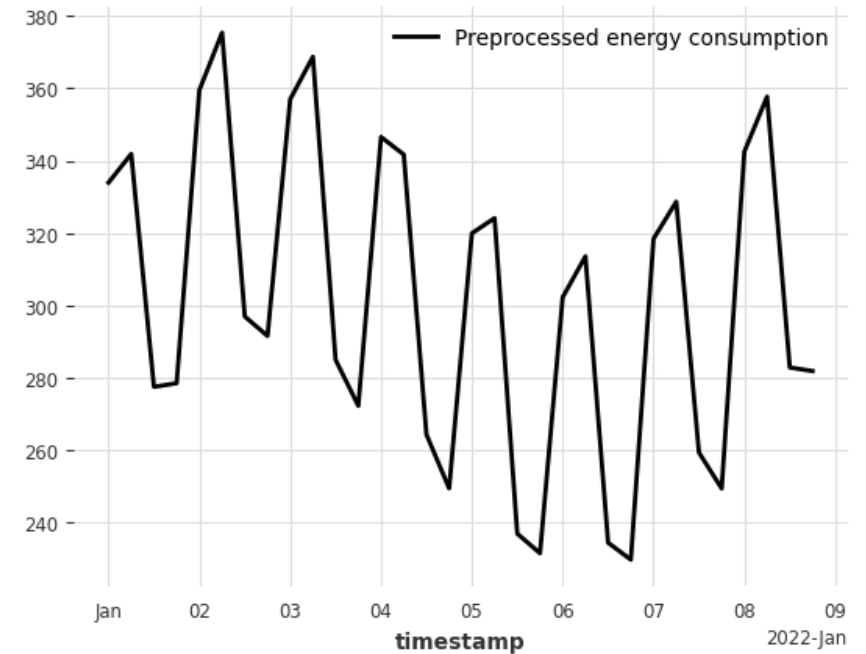
DARTS LIBRARY: PREPROCESSING

```
from darts.dataprocessing.transformers import Scaler

series = fill_missing_values(series, method='linear')

series = series.resample('6h', method='sum')

series = Scaler(StandardScaler()).fit_transform(series)
```



- **Easy** data manipulation
- Built-in **scikit-learn** methods



ARIMA WITH DARTS

```
from darts.models import ARIMA

model = ARIMA(p=2, d=1, q=2)
model.fit(train)
pred = model.predict(len(val))
```

- Darts uses a Wrapper around STASTMODELS models



DARTS LIBRARY: TRAINING

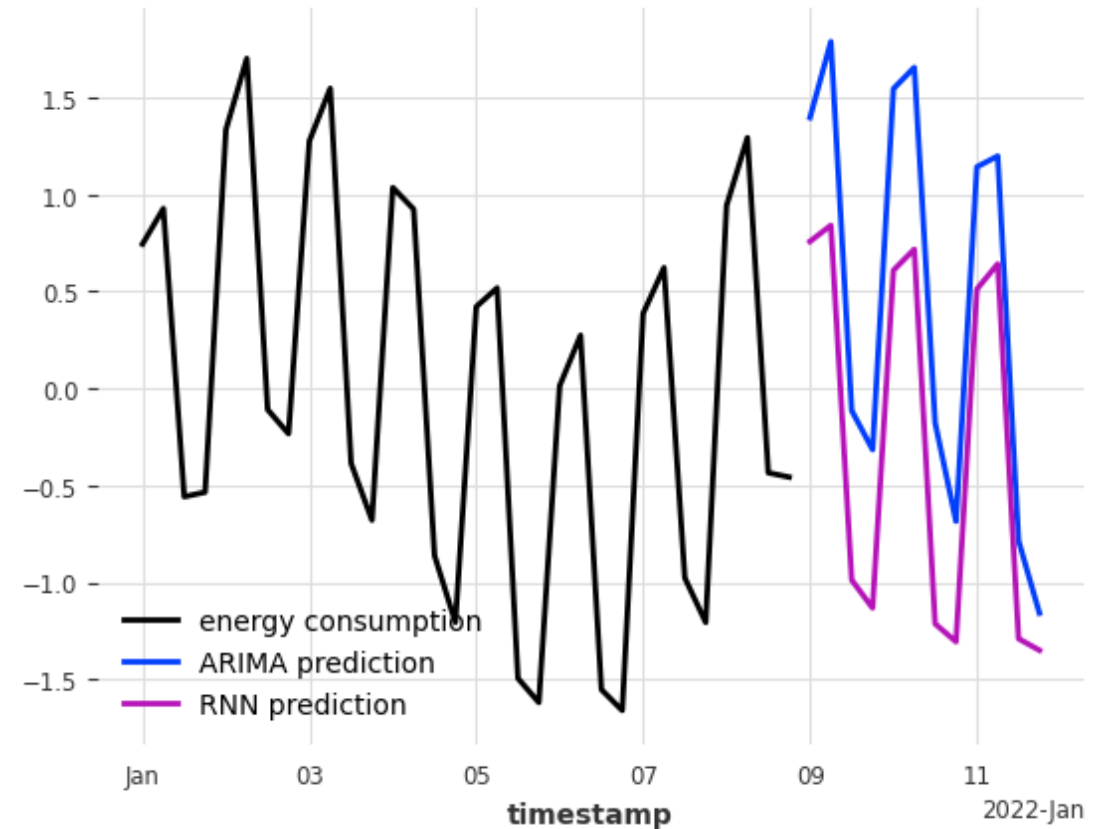
```
from darts.models import ARIMA, RNNModel

model1 = ARIMA()
model2 = RNNModel(model='RNN', input_chunk_length=12, output_chunk_length=1)

model1.fit(series)
model2.fit(series)

pred1 = model1.predict(n=12)
pred2 = model2.predict(n=12)
```

- All models share the SAME API
- Automatic creation of input/output **windows** via defining *input_chunk_length* and *output_chunk_length*



CODING SESSION

Coming up next in notebook:

- Train a model using Darts

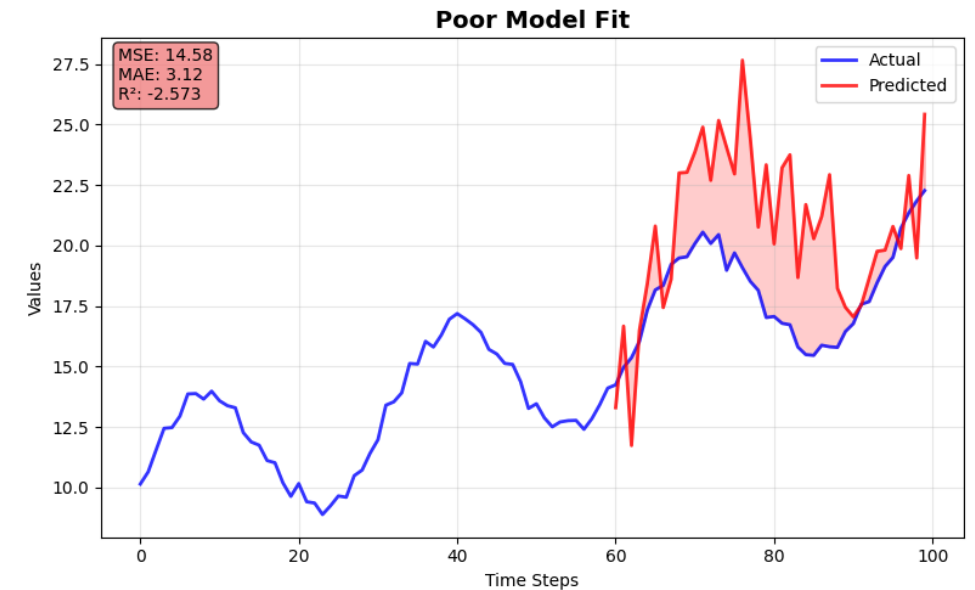


EVALUATION FOR TIME-SERIES DATA

- **Goal:** quantify how close our **forecasts** are to the **true values**
- Forecasting $\neq$ classification $\rightarrow$ we measure ***error magnitude***
- **Different models, same metric** $\rightarrow$ enables fair comparison

⚠ Important:

- Train on the past
- Test on unseen future data



EVALUATION FOR TIME-SERIES DATA

Metric	Formula	Interpretation
Mean Absolute Error (MAE)	$= \frac{1}{n} \sum_{t=1}^n y_t - \hat{y}_t $	Measures the average magnitude of the errors
Root Mean Squared Error (RMSE)	$= \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}$	- Penalizes larger errors more than MAE. - Sensitive to outliers
Mean Absolute Percentage Error (MAPE)	$= \frac{100}{n} \sum_{t=1}^n \left \frac{y_t - \hat{y}_t}{y_t} \right $	- Expresses error as a percentage of the true value - Not defined when $y_t = 0$
Symmetric Mean Absolute Percentage Error (sMAPE)	$= \frac{200}{n} \sum_{t=1}^n \frac{ y_t - \hat{y}_t }{ y_t + \hat{y}_t }$	Scale-independent and robust when values are near zero.

 Important:

- **MAE & RMSE** → same scale as data
- **MAPE / sMAPE** → relative % errors
- **sMAPE** preferred when values can approach 0



EVALUATION FOR TIME-SERIES DATA

```
from darts.metrics import mae, rmse, mape, smape

train, val = series[:-36], series[-36:]
model.fit(train)
pred = model.predict(len(val))

print("MAE:", mae(val, pred))
print("RMSE:", rmse(val, pred))
print("sMAPE:", smape(val, pred))
```

⚠ Important:

- Train on the past
- Test on unseen future data



CODING SESSION

Coming up next in notebook:

- Evaluate the models using the different evaluation metrics



PROPHET: AUTOMATIC FORECASTING BY FACEBOOK

Prophet is a model designed for **business time-series forecasting**, built to handle:

- **Strong seasonality** (daily, weekly, yearly)
- **Trend changes** (trend breaks / changepoints)
- **Holidays and special events**
- **Missing values & outliers**

PROPHET

Why Prophet?

- Very **easy to use** → minimal tuning
- Very **robust** → handles messy, real-world business data
- Highly **interpretable** → decomposable into Trend + Seasonality + Holidays
- Scales to long histories and noisy signals



PROPHET MODEL STRUCTURE

Prophet decomposes the time series into:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

1. Trend component $g(t)$

- Linear or logistic growth
- Automatic detection of **changepoints** (where trend shifts)

2. Seasonality $s(t)$

- Modeled using **Fourier series**
- Can include custom periodicities (e.g., hourly)

3. Holidays & Events $h(t)$

- Company-specific events
- Holidays (e.g., Christmas, Ramadan, Black Friday)

4. Noise ϵ_t

- Captures irregular fluctuations

*“Automatically detects frequency and trend breaks.
No need to specify order or differencing.”*



CODING SESSION

Coming up next in notebook:

- Train Prophet model

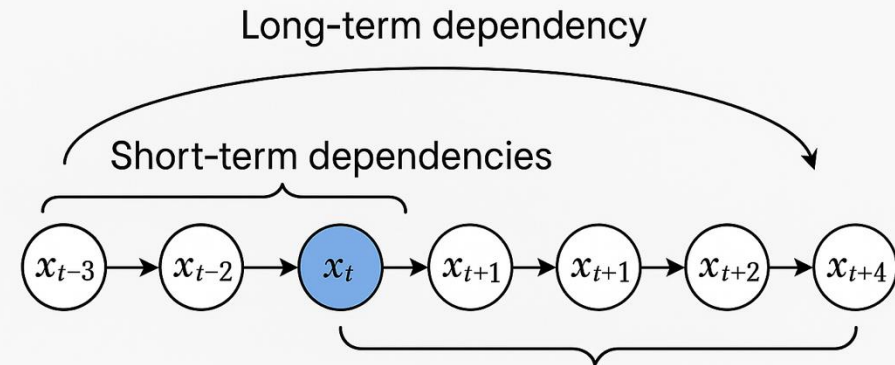


CHALLENGES

Long-term dependencies refer to situations where events or values that happened **far in the past** still have a significant influence on the **current or future** observations.

- **Why it matters:**
Traditional models (e.g., ARIMA) struggle to retain information from distant timesteps
- **Deep Learning models** were designed to capture long-term dependencies more effectively

Long-term Dependencies



Example

- an economic crisis leaves effects on markets for decades
- a drought years ago can still affect groundwater levels today

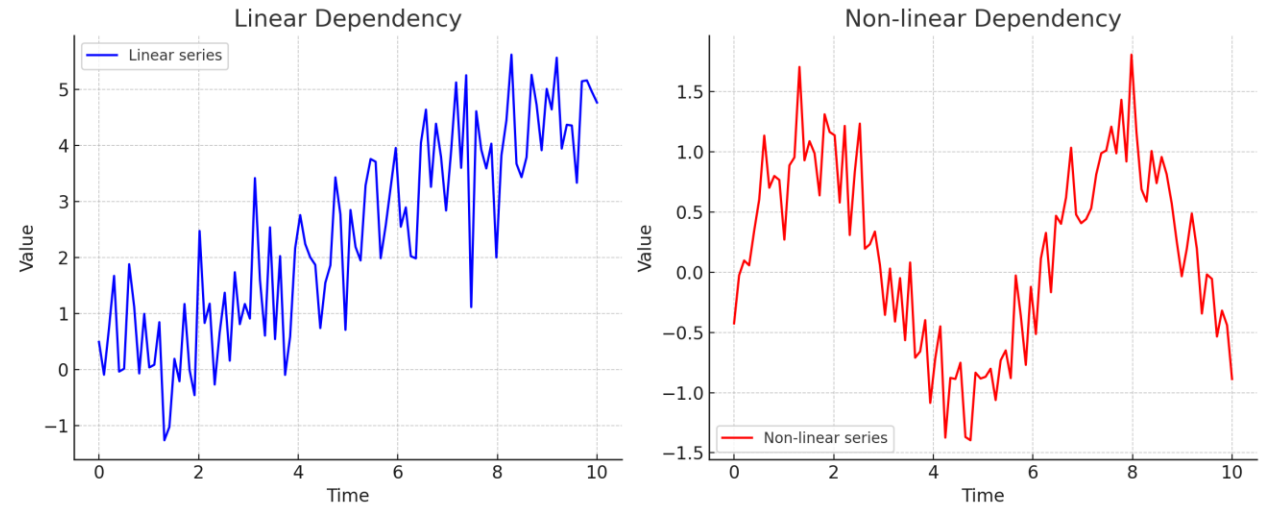


CHALLENGES

Non-linearity means that the relationship between **past** and **future** values of the series cannot be adequately described by a straight line (linear function).

$$x_t = \alpha_0 + \alpha_1 x_{t-1} + \alpha_2 x_{t-2} + \dots + \epsilon_t$$

- **Why it matters:**
Real-world time series often show **non-linear dynamics**
- **Deep learning** methods are well-suited to handle **nonlinear dynamics** in time series data, as their architectures can approximate **complex nonlinear** relationships



Example

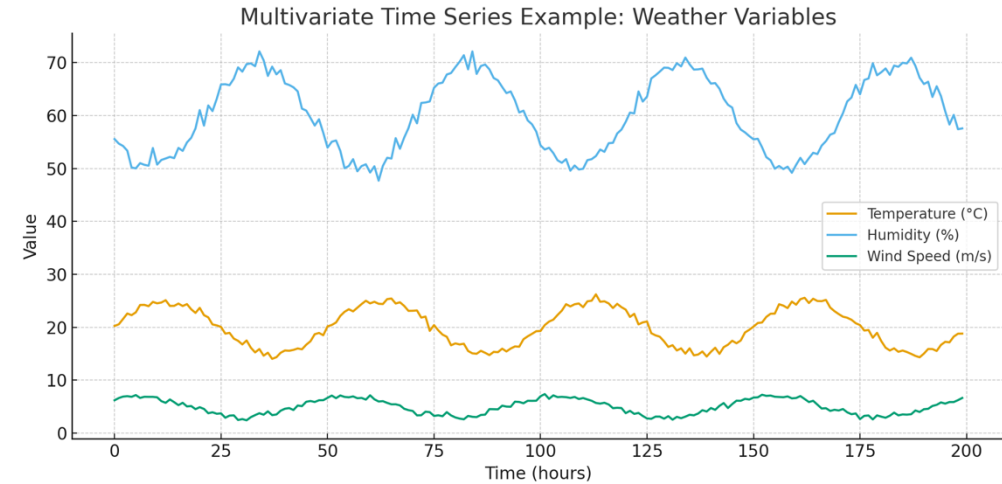
- Linear: Sales growth with a steady trend
- Non-linear: Neural signals, heart rate



CHALLENGES

Many real-world time series are **multivariate**, meaning that **multiple variables evolve together over time**, often influencing one another.

- **Why it matters:** Traditional models often treat each variable **independently**, or assume **simple linear relationships** between them
- **Deep Learning** models can jointly model **interactions across exogenous series**



Example

- Weather forecasting → temperature, humidity, pressure, wind
- Finance → prices, trading volume, interest rates



WHEN TO USE DEEP LEARNING

✅ Cases where deep learning is **worth** it :

- **High-dimensional** multi-series data
- **Nonlinear complex** data
- **Big** data and **large** histories

❌ Cases where deep learning is **not worth** it :

- **Small** dataset, **short** time series
- Mostly **linear** + **seasonal** data
- **Interpretability** is more important



COFFEE BREAK



WINDOWING

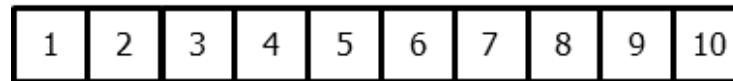
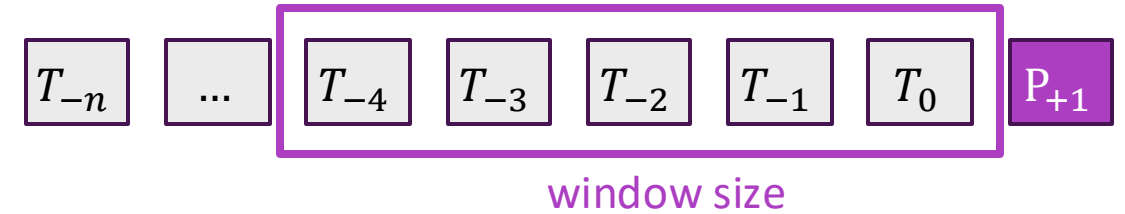
Windowing is the technique of **chunking** the data into fixed-size segments (windows) to analyze or model local patterns

$$W_k = \{x_t, x_{t+1}, \dots, x_{t+w-1}\}, \quad t = 1 + (k - 1)s$$

w = window size

s = step size (stride)

k = window index (1, 2, ...)



WINDOWING

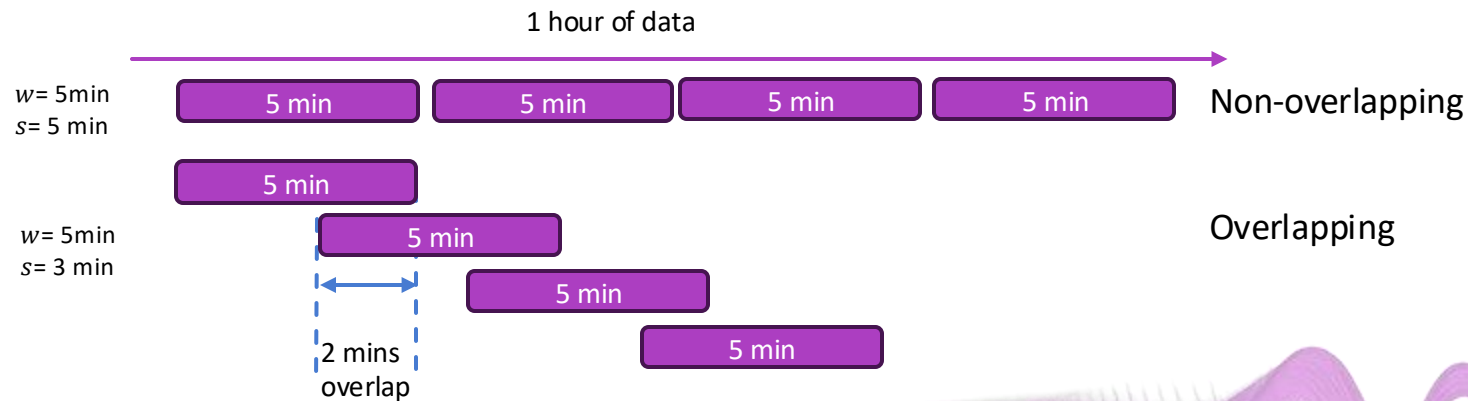
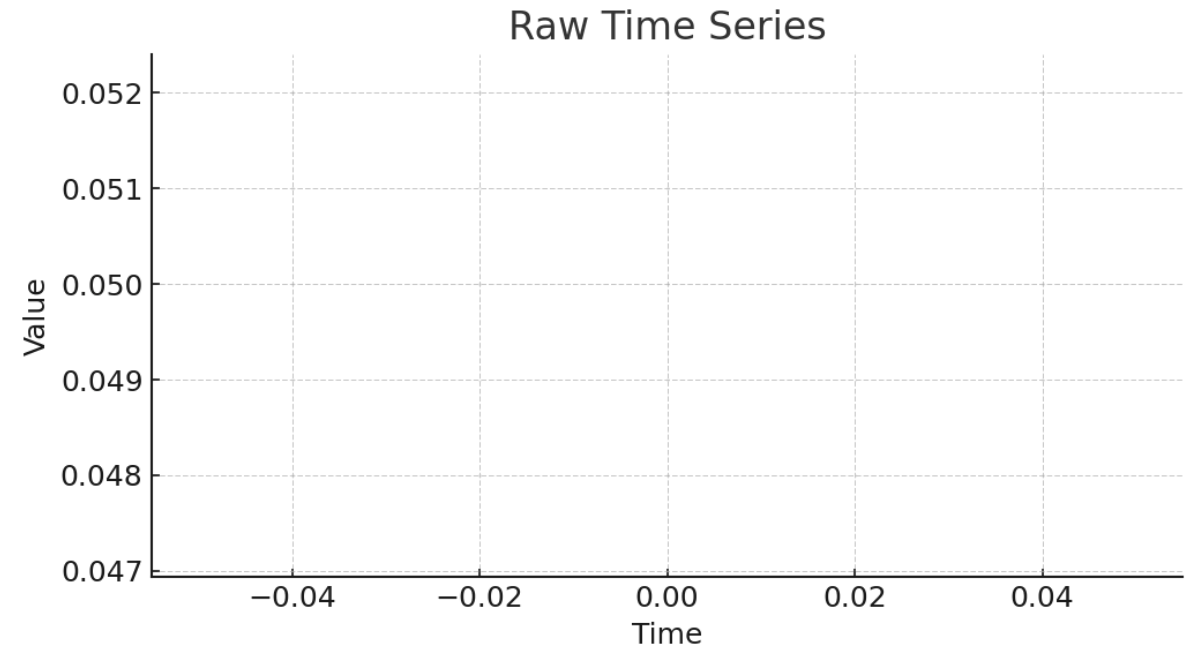
Windowing is the technique of **chunking** the data into fixed-size segments (windows) to analyze or model local patterns

$$W_k = \{x_t, x_{t+1}, \dots, x_{t+w-1}\}, \quad t = 1 + (k - 1)s$$

w = window size

s = step size (stride)

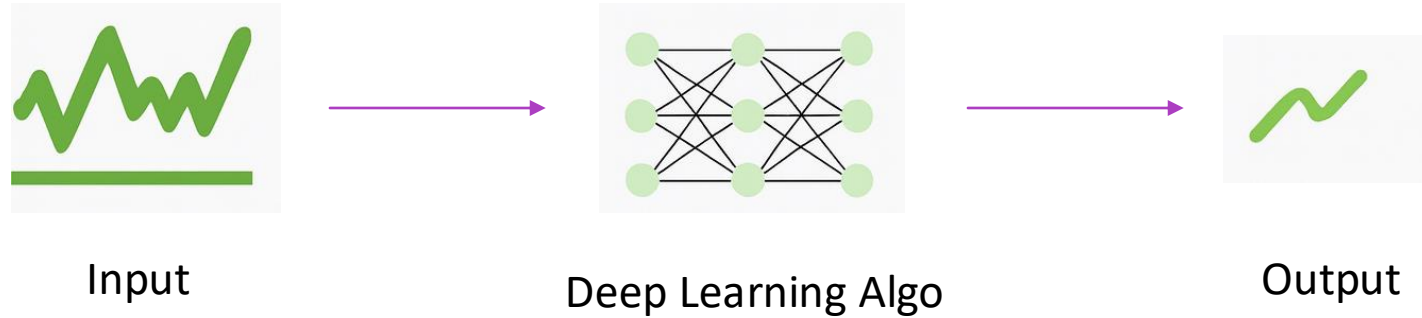
k = window index (1, 2, ...)



Why?

- Captures local dynamics
- Feature extraction & rolling stats
- Forecasting setup

DEEP LEARNING VS MACHINE LEARNING/ STATISTICS



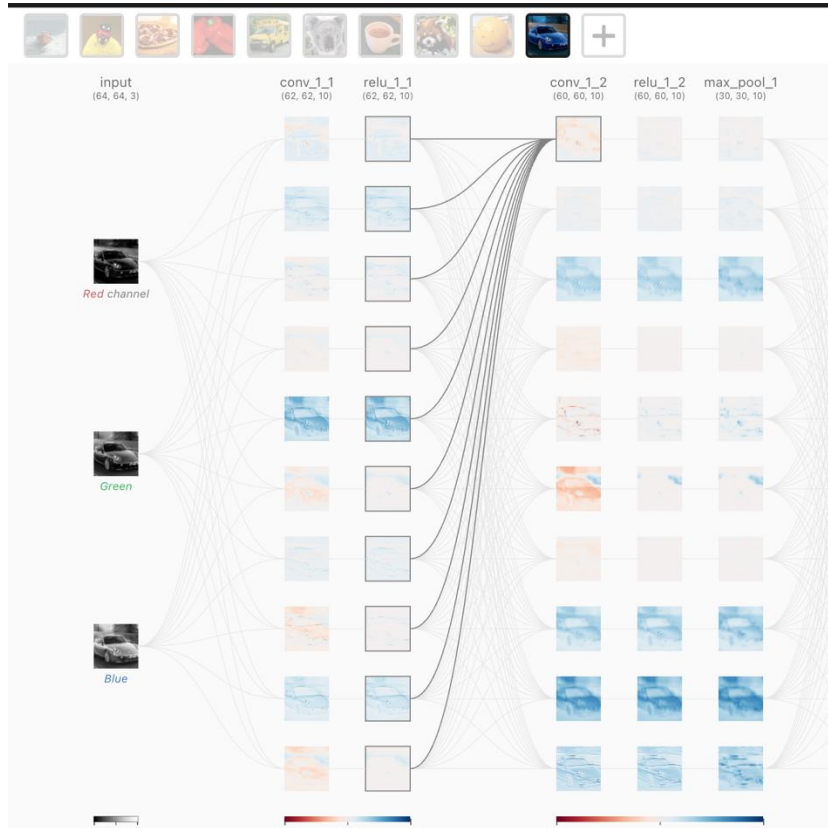
It learns and adapts with little to no human intervention



Needs a lot of human intervention to prepare the data and extract useful features



DEEP LEARNING

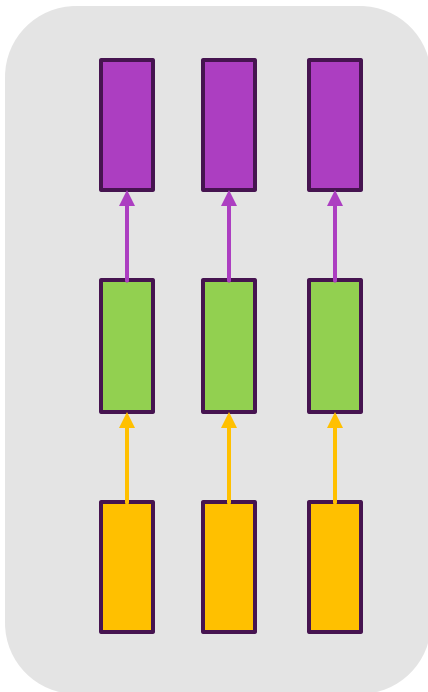


Let's dive into a deep learning model!



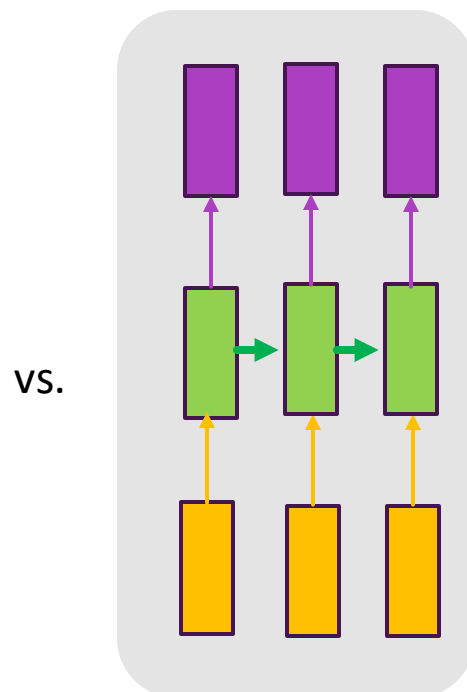
WHY RECURRENT NEURAL NETWORKS?

Feedforward
Neural Network



⚠ no memory

Recurrent
Neural Network

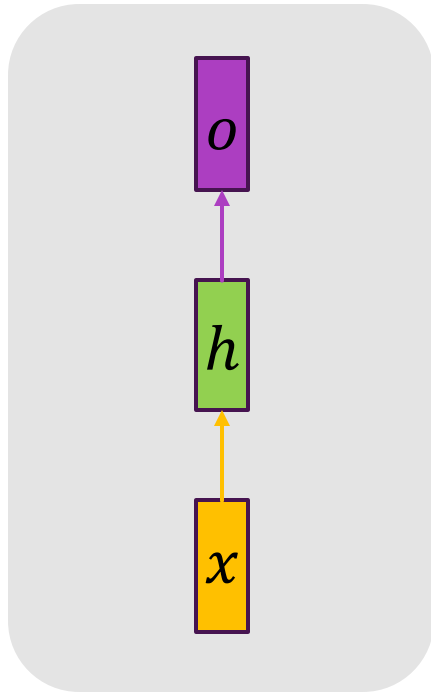


⚠ temporal memory

- Standard NNs treat each input **independently**
→ no sense of time or order.
- But in time series, **the past matters**:
 - Stock prices depend on previous days.
 - Energy consumption depends on the time of day or season.
- We need models that can **remember** and **connect across time** → RNNs.

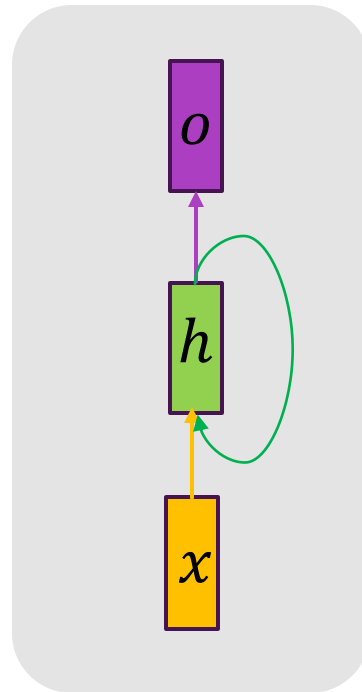


RECURRENT NEURAL NETWORKS: HIDDEN STATE



Feedforward
Neural Network

vs.

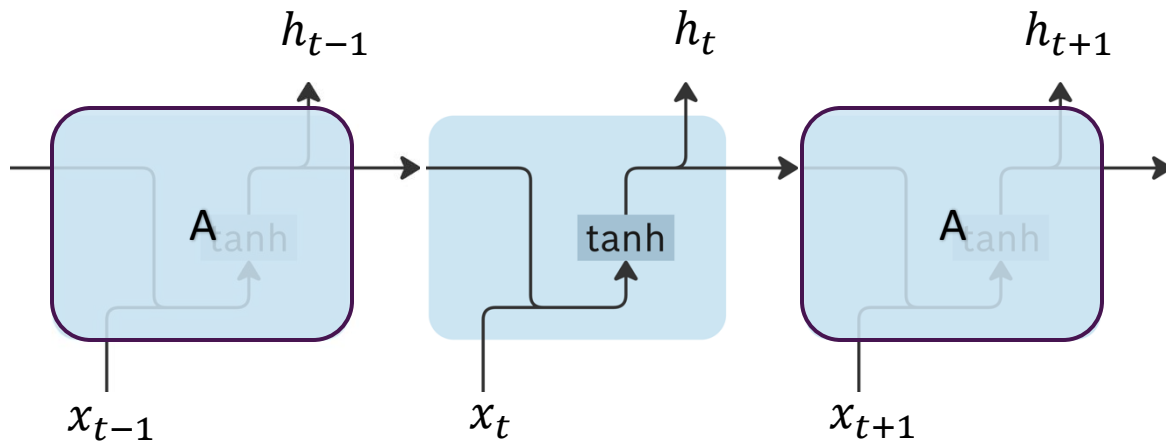


Recurrent
Neural Network

- RNNs process **sequences** of data while keeping a **memory of previous inputs**
- **Hidden state** is a vector which encapsulates useful **information** obtained **from previous cells** given
- Unlike feedforward NNs, they consider **input order and context**
- Ideal for **sequence-dependent tasks** (e.g., speech, text, time series)



RECURRENT NEURAL NETWORKS: UNROLLED



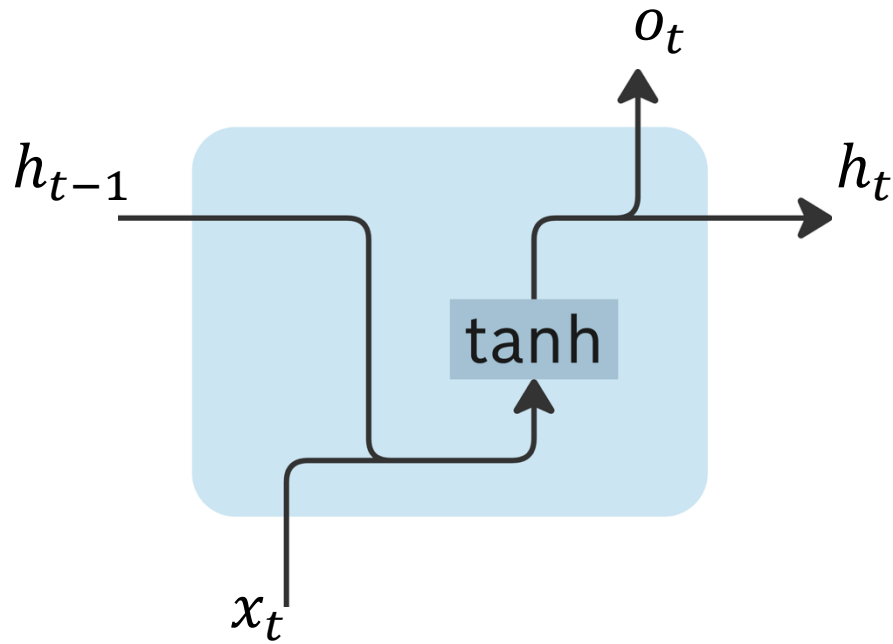
- When applied over time, the same cell repeats -> sharing weights.
- This “unrolled” view shows how information flows through time steps.

Key takeaway:

“The same weights are used at every time step -> that’s how it learns temporal patterns.”



RECURRENT NEURAL NETWORKS: CELL VIEW



- RNNs process **sequences** step by step.

Each time step has:

- Current input: x_t
- Previous hidden state: h_{t-1}
- New hidden state: h_t

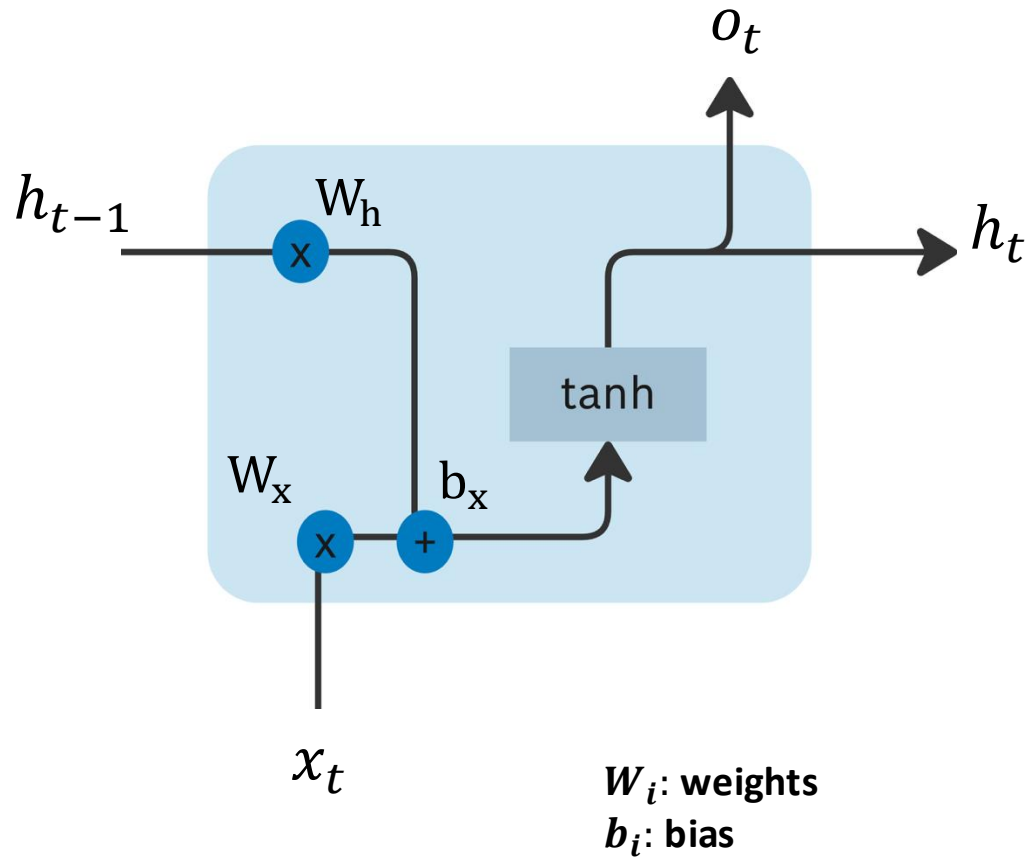
The hidden state acts as **memory** of past inputs.

Key takeaway:

"An RNN learns from both the current input and what it remembers from before."



RECURRENT NEURAL NETWORKS: CELL VIEW



- RNNs process **sequences** step by step.

Each time step has:

- Current input: x_t
- Previous hidden state: h_{t-1}
- New hidden state: h_t

The hidden state acts as **memory** of past inputs.

Key takeaway:

"An RNN learns from both the current input and what it remembers from before."

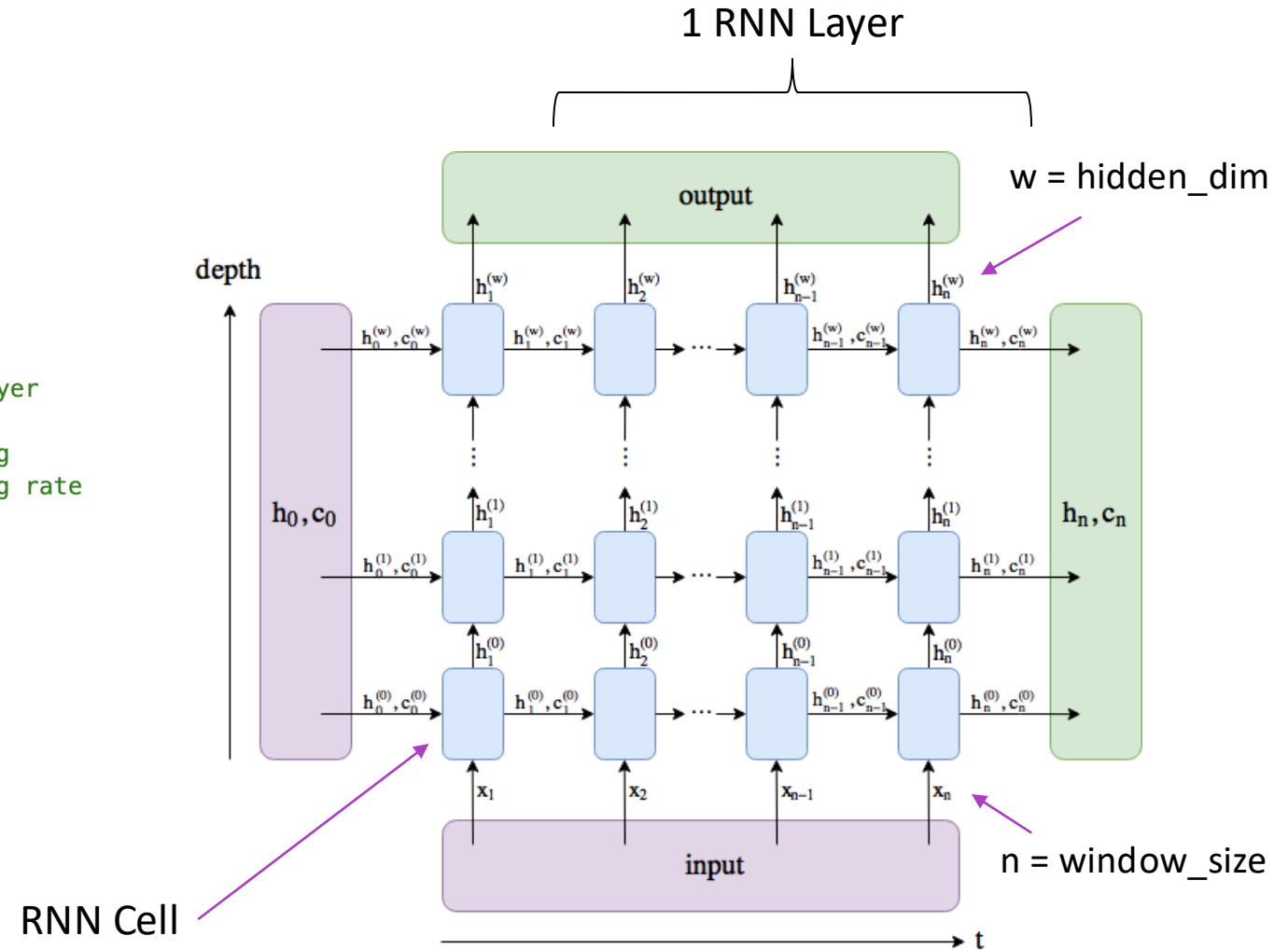


RNN PARAMETERS

```
from darts.models import BlockRNNModel

model_darts_ex = BlockRNNModel(
    model="RNN",                # Use vanilla RNN cells
    input_chunk_length=window_size, # 24 timesteps lookback
    output_chunk_length=forecast_horizon, # 1 timestep forecast
    hidden_dim=50,               # 50 hidden units per layer
    n_rnn_layers=2,              # 2 stacked RNN layers
    batch_size=256,              # Batch size for training
    activation="adam",           # Adam optimizer learning rate
    pl_trainer_kwargs={"accelerator": "gpu", "max_epochs": 50},
    random_state=42,
)
```

- `window_size` controls *how far back the network looks*
- `hidden_dim` controls *how much the network can remember (capacity)*
- `n_layers` = stacked layers correspond to “depth”
- `output_chunk_length` controls the forecast horizon



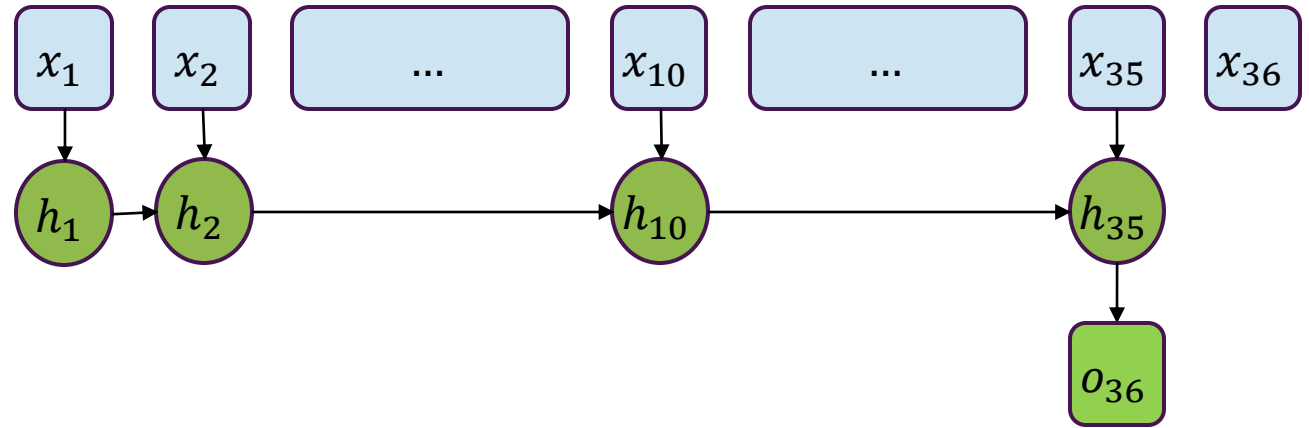
HOW FAR DOES AN RNN LOOK BACK?

Scenario:

We have a time series

$$x_1, x_2, \dots, x_{35}$$

and we want to predict x_{36} .



Key idea:

At each step, it updates a *hidden state* h_t that carries information from all previous inputs.

By the time we reach step 35:

$$h_{35} = f(x_{35}, h_{34}) = f(f(\dots f(x_1, h_0) \dots))$$

So h_{35} is a **summary of everything up to step 35**.



VANISHING/ EXPLODING GRADIENTS

RNNs use **Backpropagation Through Time (BPTT)** to update weights.

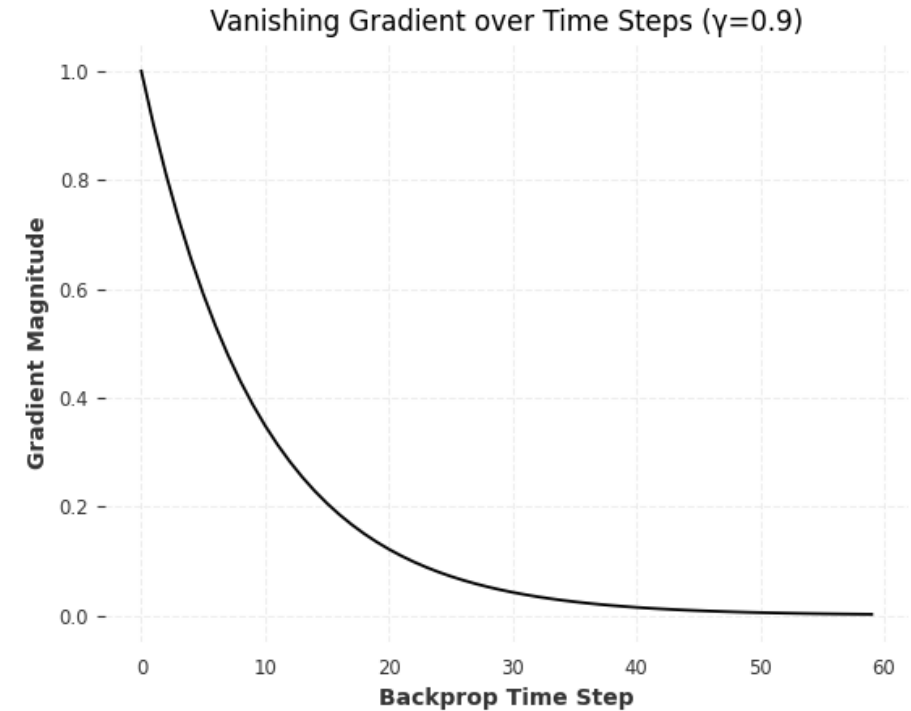
Gradients are multiplied at each time step →

If < 1 : they **vanish** → earlier steps forgotten

If > 1 : they **explode** → unstable training

Leads to:

- ✗ No learning of long-term dependencies (vanishing)
- ✗ Diverging weights / NaNs (exploding)



Key takeaway:

“Standard RNNs struggle to learn long-term dependencies because gradients fade or explode during backpropagation.”



CODING SESSION

Coming up next in notebook:

- Train first rnn model



QUIZ TIME

Q1. What is the role of the **hidden state** in an RNN?

✓ **It represents the network's memory of past inputs.**

Q2. What happens when we “**unroll**” an RNN across time?

✓ **We visualize the same cell repeated with shared weights over multiple steps.**



HOW TO IMPROVE THE FORECAST?

Time series										X					y
1	2	3	4	5	6	7	8	9	10	1	2	3	4	5	6
										2	3	4	5	6	7
										3	4	5	6	7	8
										4	5	6	7	8	9
										5	6	7	8	9	10



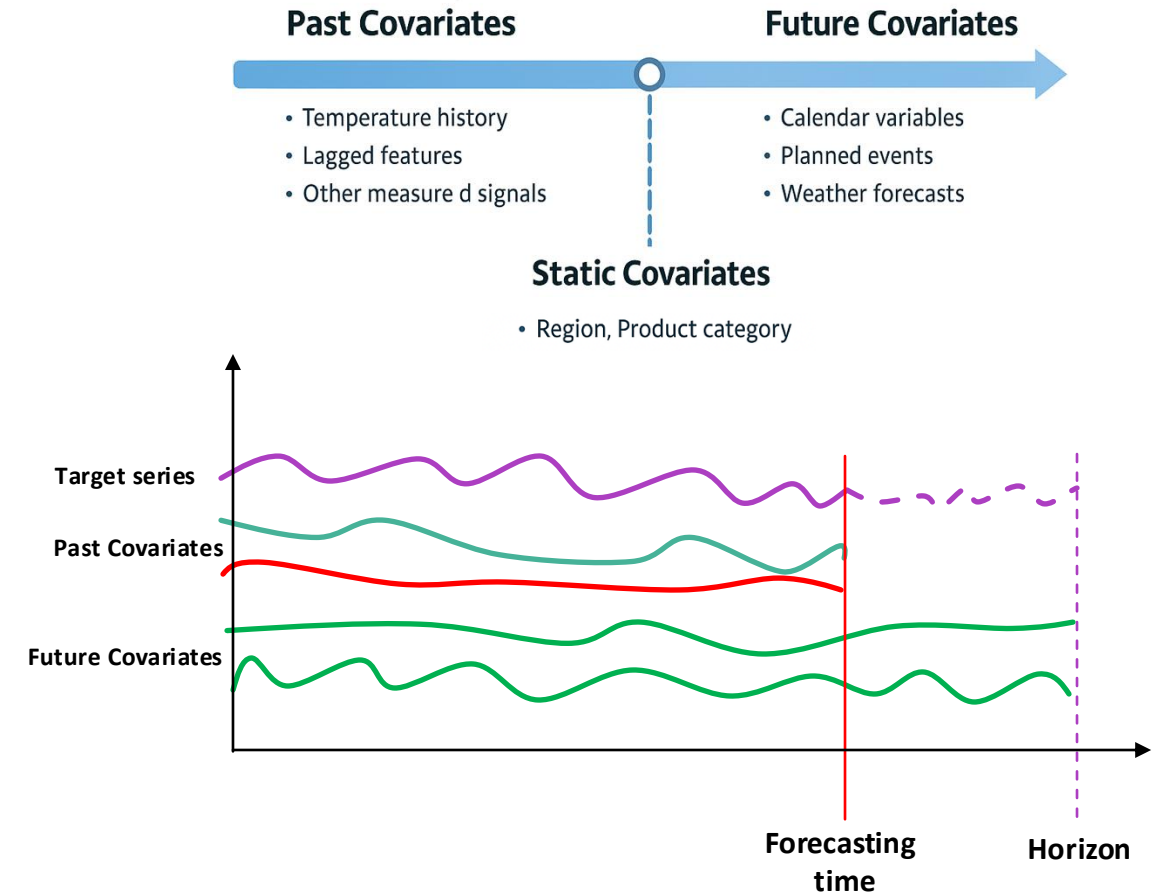
WHAT ARE COVARIATES?

Covariates are **additional** explanatory time series that **help** improve forecasts. They can be:

- **Past covariates:** known only **up to** the current time (e.g., past weather)
- **Future covariates:** **known** or **predictable** into the **future** (e.g., holidays, planned events)
- **Static covariates:** **global attributes** that don't change with time.

They provide context about what **influences** the target variable.

- Temperature → affects electricity demand
- Promotions → affect sales



LUNCH BREAK: 12:25-13:25



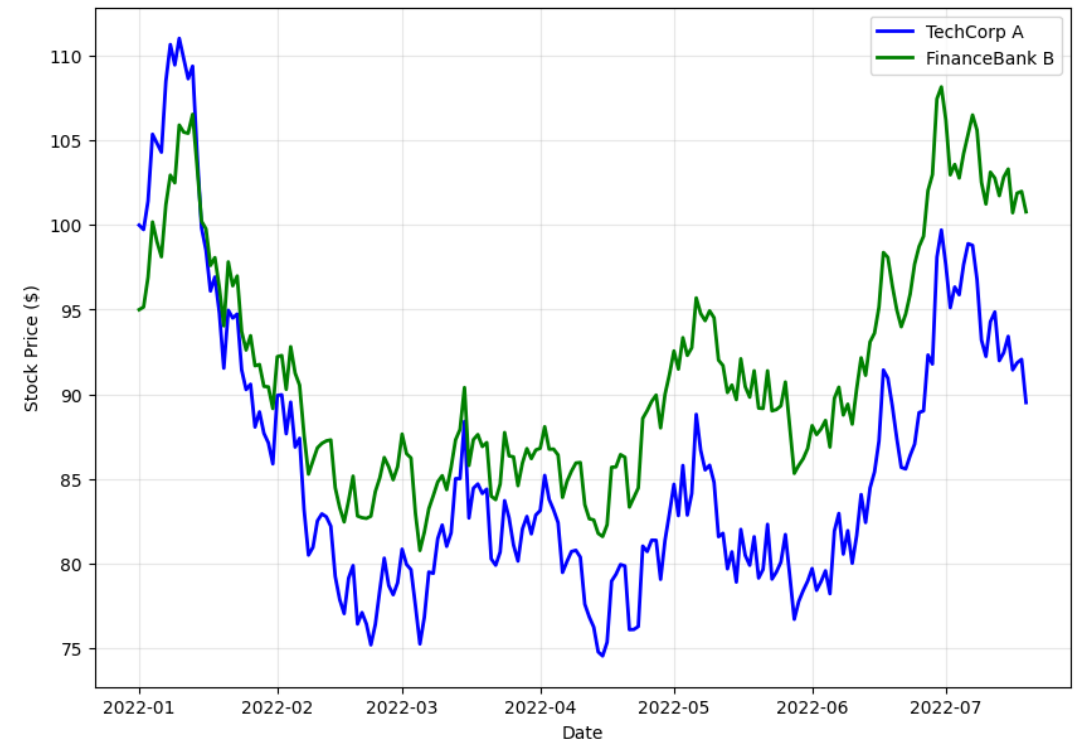
CORRELATION IN TIME SERIES

Correlation measures the strength and direction of a relationship between two variables.

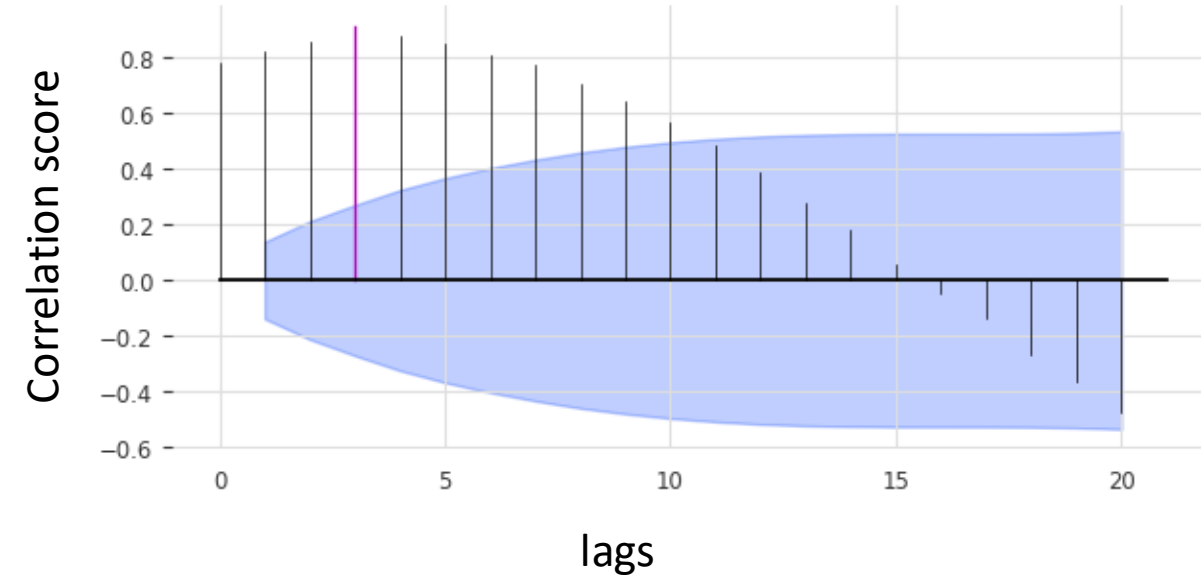
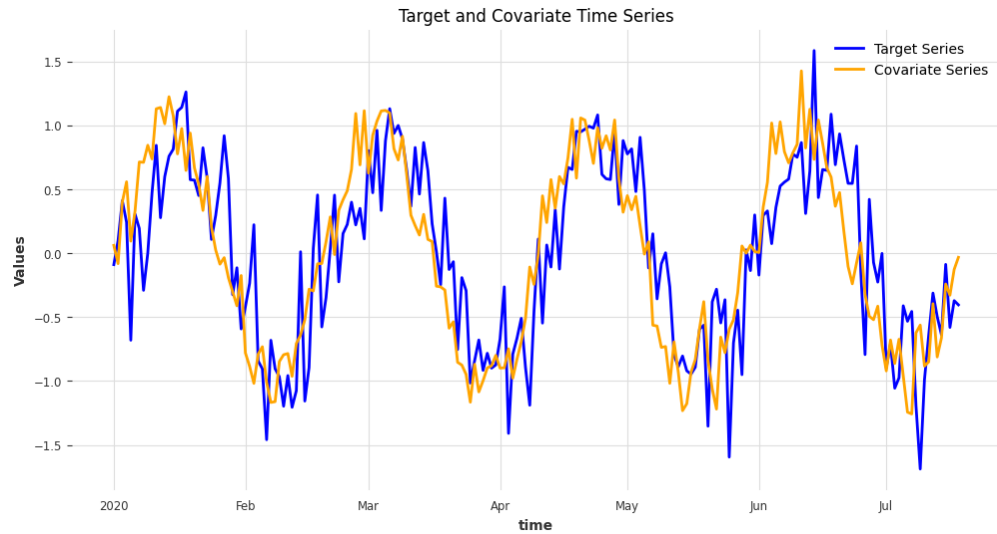
$$-1 \leq \text{Correlation score} \leq 1$$

- 1: perfect positive correlation
- -1: perfect negative correlation
- 0: no linear relationship

In time series, correlation helps identify **which covariates influence** the target.



(CROSS-) CORRELATION USING DARTS



```
# analyze correlations target vs covariates
from darts.utils.statistics import plot_ccf

plot_ccf(target_series[tr_column_1],
          past_covariates[cov_column_1],
          max_lag=20)
```



CODING SESSION

CASE STUDY: ELECTRICITY CONSUMPTION

Measurements: electric power consumption of
households & SMEs (low voltage)

businesses & services (medium voltage)

Frequency: 15 minutes sampling rate

Unit: kW

Values between **2015-01-01** and **2022-08-31**.

Source: https://data.stadt-zuerich.ch/daxtaset/ewz_stromabgabe_netz_ebenen_stadt_zuerich



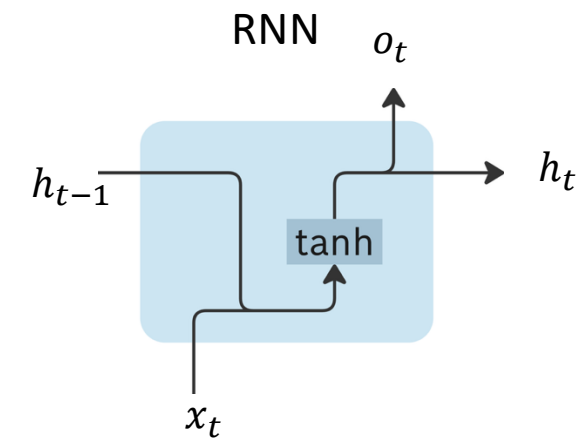
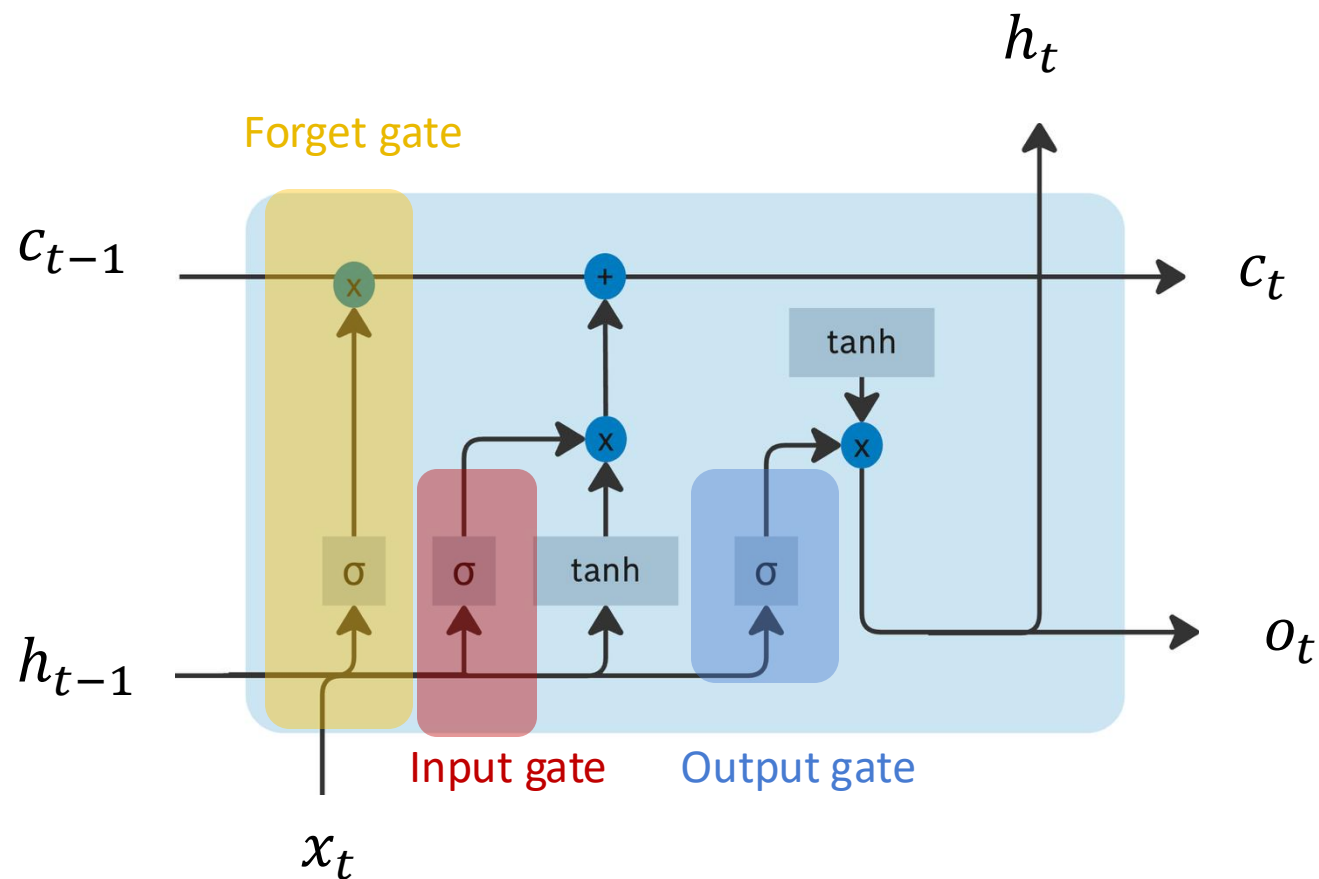
CODING SESSION

Coming up next in notebook:

- Train first rnn model using past and future covariates



LONG SHORT-TERM MEMORY



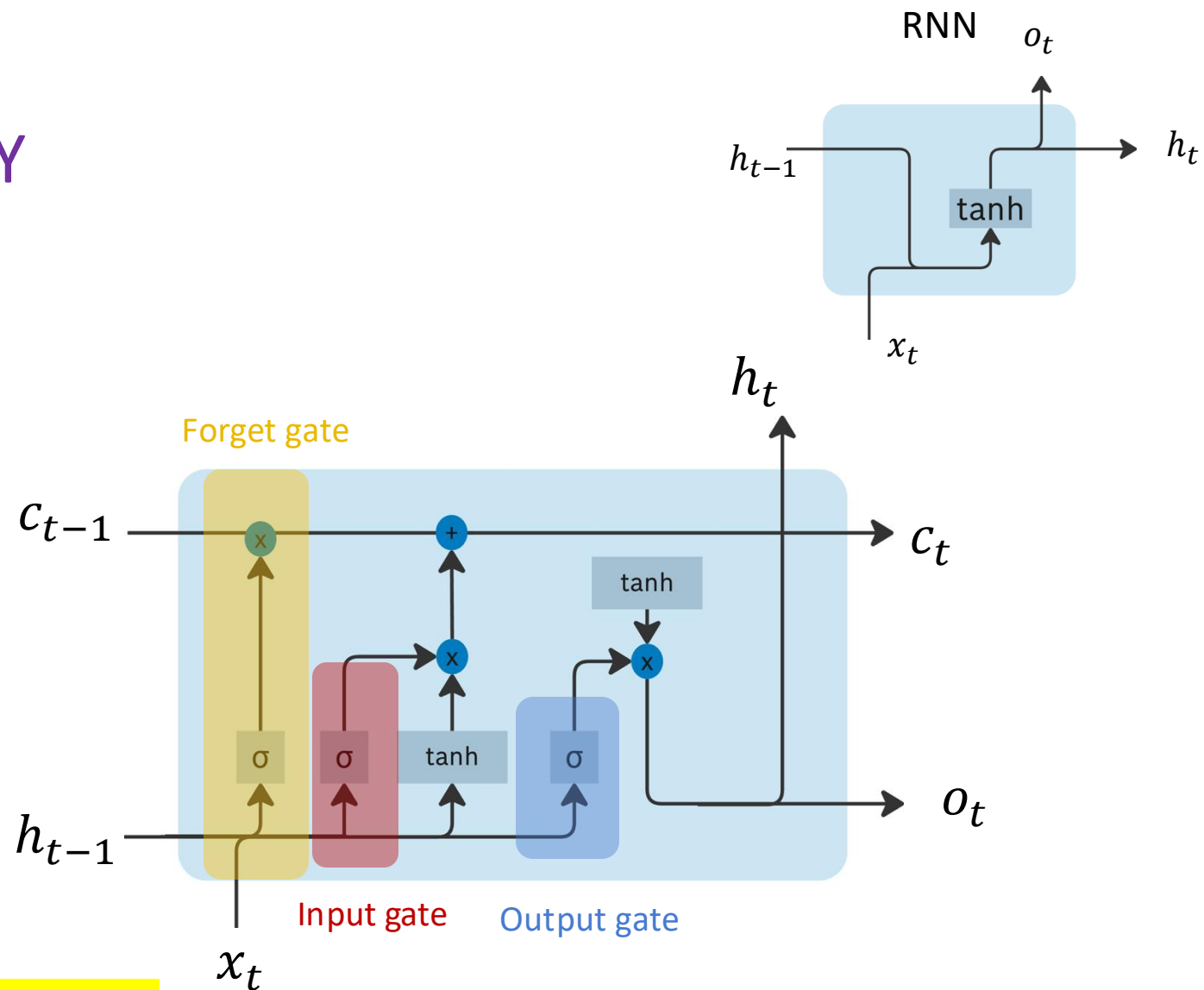
LONG SHORT-TERM MEMORY

LSTMs introduce a **memory cell** c_t , which keeps important information over long sequences.

Used gates:

- **Forget Gate:** decides what to erase from memory
- **Input Gate:** decides what new info to store
- **Output Gate:** decides what to send out

“Learns what to remember, what to forget, and what to output”

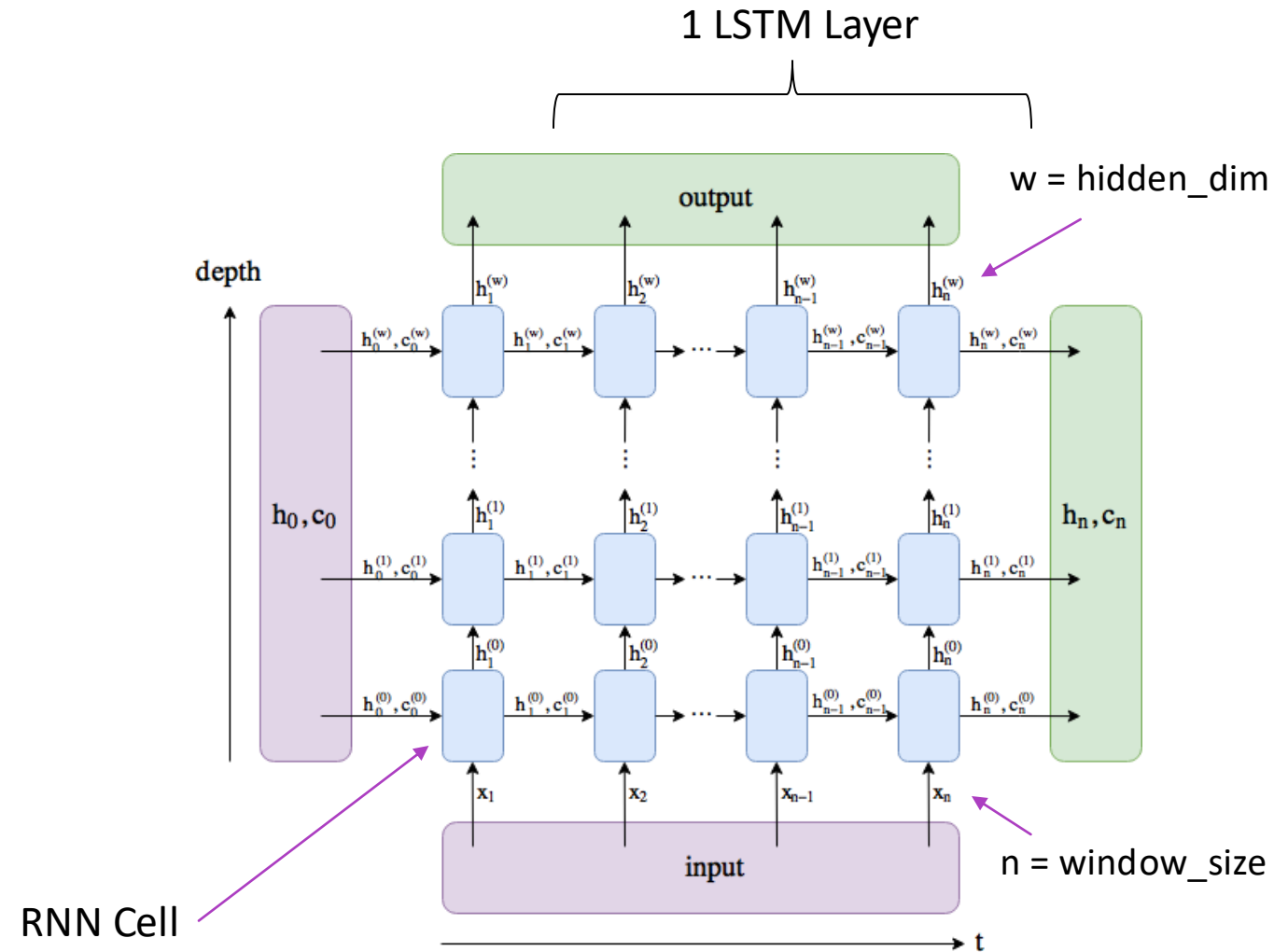


LSTM PARAMETERS

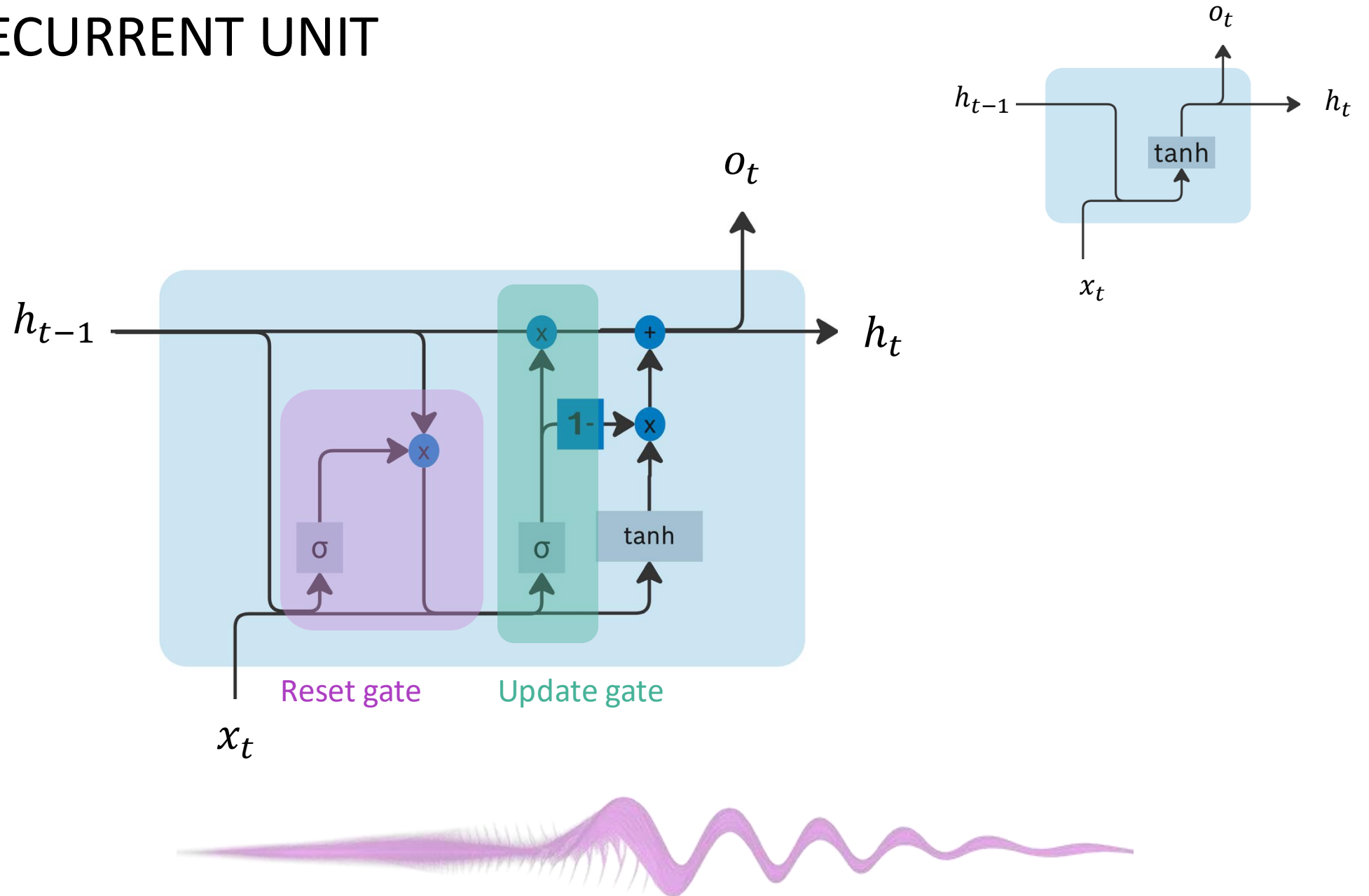
```
from darts.models import BlockRNNModel

model_darts = BlockRNNModel(
    model="LSTM",
    input_chunk_length=window_size,
    output_chunk_length=forecast_horizon,
    hidden_dim=100,
    n_rnn_layers=2,
    batch_size=64,
    activation="adam",
    pl_trainer_kwargs={"accelerator": "gpu"},
    random_state=42,
)
```

- `window_size` controls *how far back the network looks*
- `hidden_dim` controls *how much the network can remember (capacity)*
- `n_layers` = stacked layers correspond to “depth”
- `output_chunk_length` controls the forecast horizon



GATED RECURRENT UNIT



GATED RECURRENT UNIT

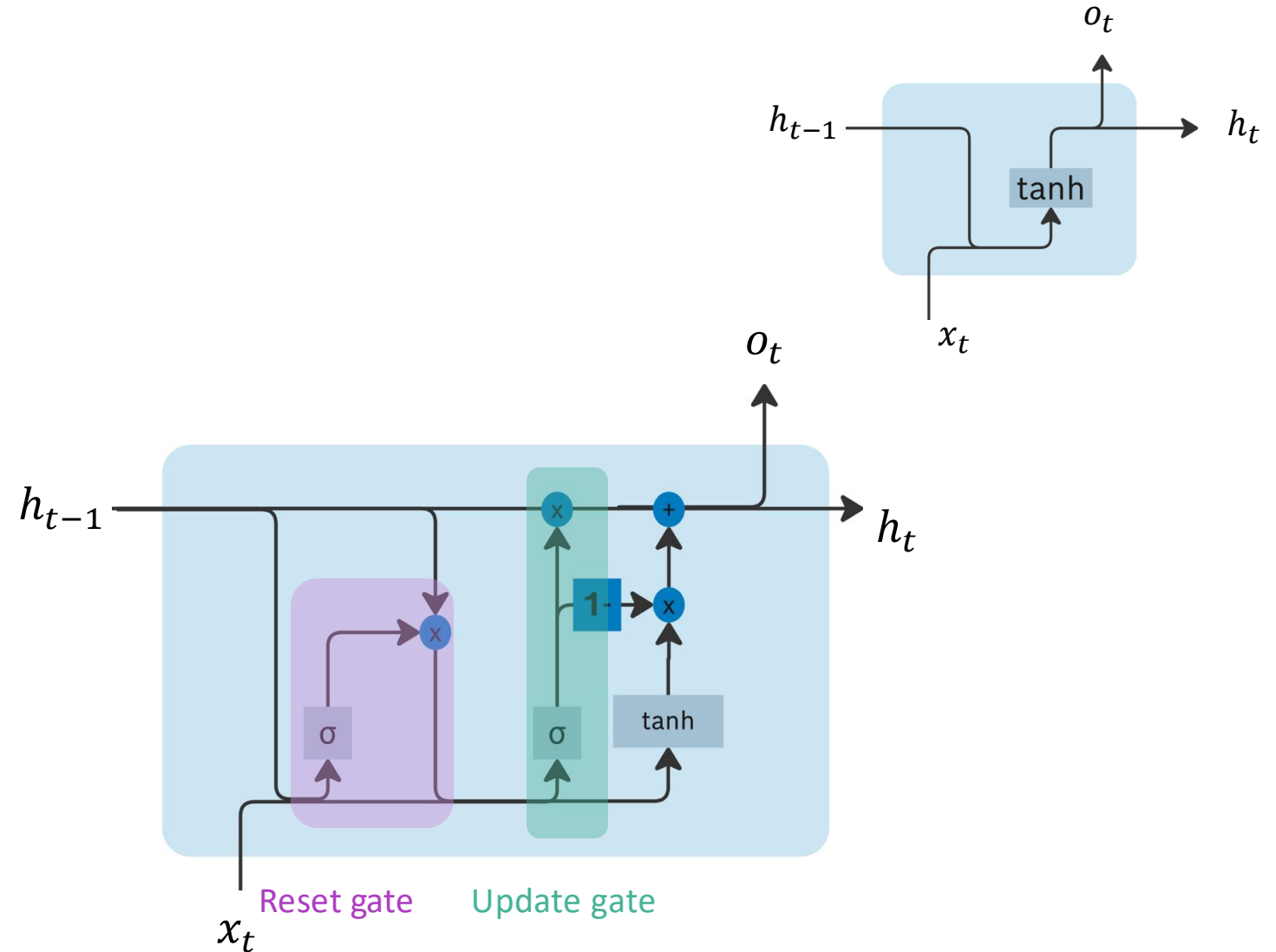
Simplified version of LSTM -> fewer gates,
fewer parameters

Used gates:

- **Reset Gate:** decides how much of the past to keep
- **Update Gate:** decides how much of the past to forget

No separate cell state -> hidden state acts
as both memory and output

Faster training and less memory use



CODING SESSION

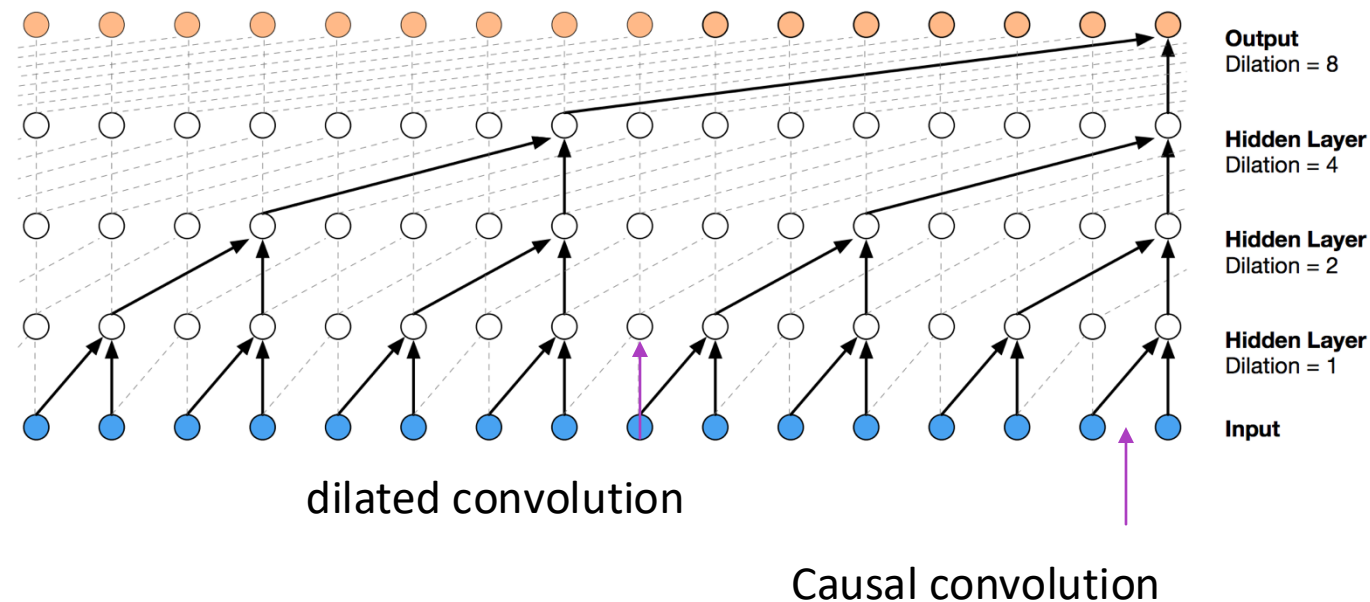
Coming up next in notebook:

- Train LSTM and GRU model



TEMPORAL CONVOLUTIONAL NETWORKS

- **Convolutional approach** to sequence modeling
-> replaces recurrence with 1D convolutions.
- **Causal convolutions:** ensure predictions only depend on *past* values.
- **Dilated convolutions:** allow the model to capture **long-term dependencies** efficiently.
- **Parallelizable** (unlike RNNs) → faster training on GPUs.
- Often **outperforms GRU** on long-sequence forecasting tasks.



"TCNs use convolution instead of recurrence — they see the past through wider, overlapping windows and capture long-term patterns efficiently."



TEMPORAL CONVOLUTIONAL NETWORKS

```
from darts.models import TCNModel

model = TCNModel(
    input_length=30,      # 📌 how many past time steps the model sees (context window)
    output_length=7,      # 📌 how many future steps the model predicts (forecast horizon)
    kernel_size=3,        # 📌 size of convolution filter (how many neighbors are seen per layer)
    num_filters=16,       # 📌 number of filters / feature maps per convolutional layer
    dilation_base=2,       # 📌 how spacing between filter taps grows in deeper layers
    dropout=0.1,          # 📌 regularization – prevents overfitting
)
```

Input_length: The TCN doesn't remember like an RNN, instead, it sees a moving window of the past.

kernel_size: Use 3–5 for shorter series, up to 7–9 for smoother long-term patterns

Num_filters: 8–32 filters is typical for small to medium datasets

dilation_base=2: each layer sees twice as far into the past.



QUIZ TIME

Q1. What main problem does the **LSTM** solve compared to a standard RNN?

✓ **Slow training speed.**

Q2. How is a **GRU** different from an LSTM?

✓ **It merges the cell and hidden state, using only two gates.**

Q3. How do **Temporal Convolutional Networks (TCNs)** handle sequence data differently?

✓ **They use 1D convolutions instead of recurrence.**

RNNs → remember short-term

LSTMs → fix long-term memory

GRUs → simplify the process

TCNs → replace recurrence with convolution



KEY TAKEAWAYS

- **Good forecasting starts with data** cleaning, resampling, normalization matter.
- Searching and adding (past/Future) **Covariates** can improve the model performance.
- **Model choice follows data complexity:**
 - Small, linear → ARIMA / Regression
 - Long, nonlinear → RNN / LSTM / GRU
 - Large-scale, multivariate → TCN / Transformer
- **Classical models:** interpretable, fast, but limited for complex dependencies and high-frequency data
- **Deep models:** learn nonlinear and long-term relationships automatically
- **Modern trend:** attention-based models for interpretable deep forecasting.



LET'S CONNECT!



Dr. -Ing. Mohamed-Ali Tnani

Time-Series & Cloud Architecture Expert

Email: Mdali.tnani@gmail.com

[Click me!](#)

